

18-799: Applied Computer Vision

Spring 2026

Feature Detection

Objectives

In today's class we will learn:

- Why do we need feature detection
- Edge detection

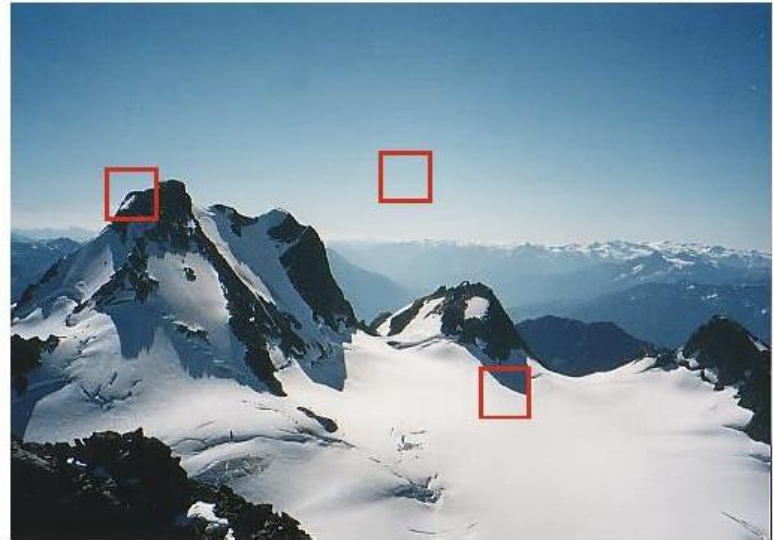
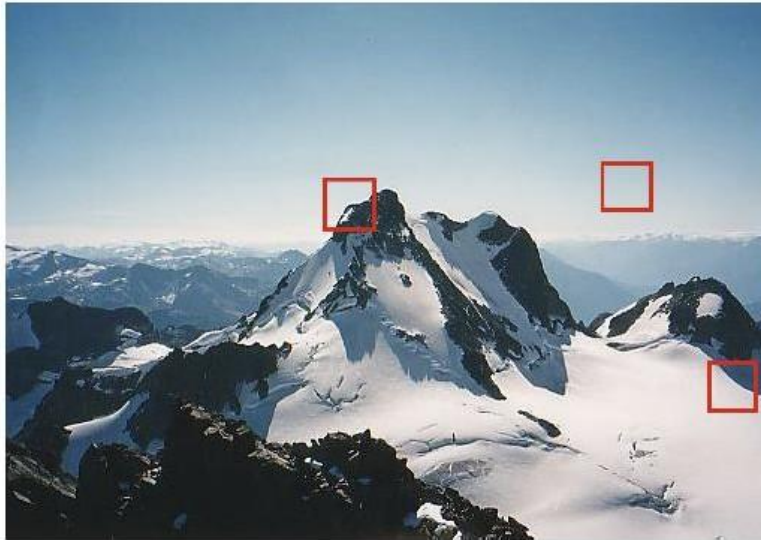
Motivation

- **Problem:** Images contain millions of pixels, but most are redundant or uninformative.
- Processing every pixel is:
 - computationally expensive
 - sensitive to noise and illumination
 - unnecessary for most vision tasks
- Vision algorithms need **a small set of reliable reference points**, not all pixels.

Motivation

- Many core vision tasks rely on **correspondences**:
 - image matching & stitching
 - tracking across frames
 - 3D reconstruction
- All of them need points that can be:
 - detected again
 - localized precisely
 - matched reliably
- **This motivates feature detection.**

Interest Points



Flat surfaces are spread over a lot of area. It is difficult to find the exact location of these patches.

Edges are much more simple. You can find an approximate location, but exact location is still difficult.

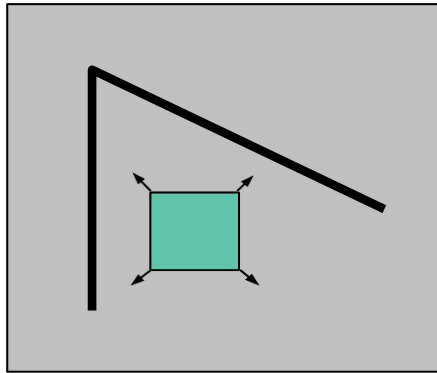
Corners can be “easily” found.

Interest Points

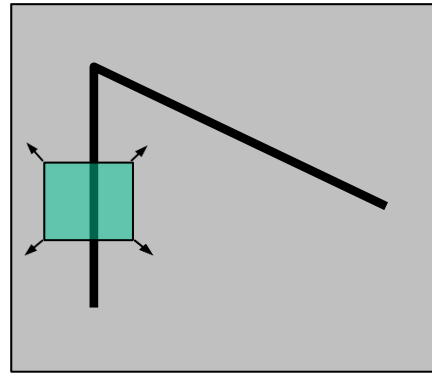
-
- Feature detection seeks points that are:
 - **repeatable** across images
 - **distinctive** from their neighbors
 - **well localized**
 - **stable** under small noise

Finding interest points in an image

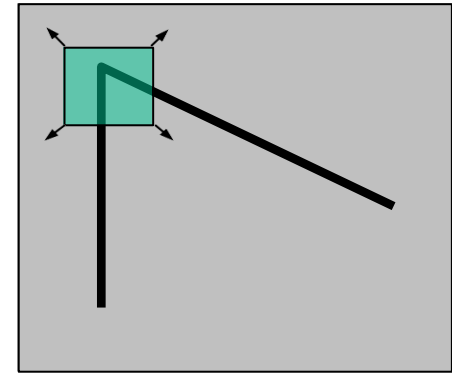
- How does the window change when you shift it?



“flat” region:
no change in
all directions



“edge”:
no change along
the edge
direction

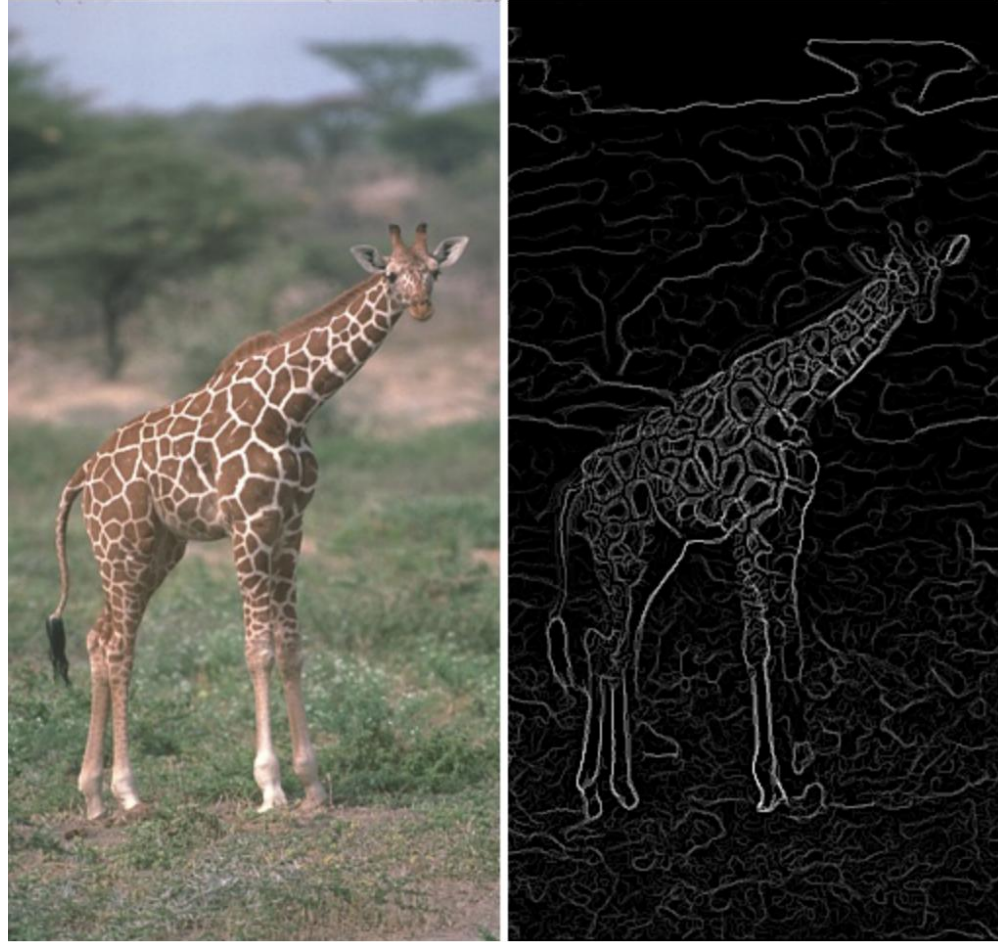


“corner”:
significant change in
all directions, i.e.,
even the minimum
change is large

Find locations such that the minimum **change** caused by shifting the window in any direction is large

What is an Edge

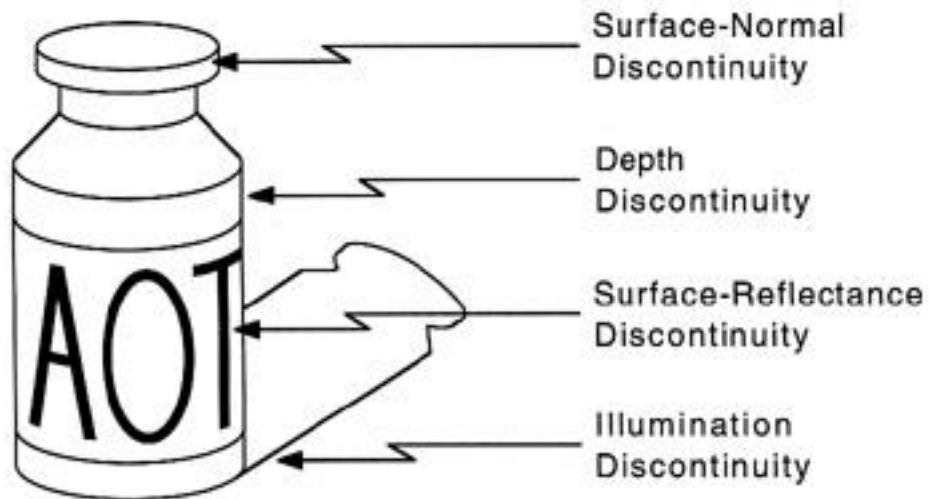
- Rapid change in image intensity within small region



<https://cs.brown.edu/courses/cs143/2013/proj5/>

What is an Edge

Rapid changes in image intensity are caused by various physical phenomena.



Source: <https://cs.stanford.edu/people/eroberts/courses/soco/projects/1997-98/computer-vision/edges.html>

What is an Edge

- An edge is a location where:
 - image intensity changes rapidly
 - the gradient magnitude is large
- If $I(x, y)$ is the image:
 - edges occur where $\nabla I = \left[\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y} \right]$ is large

Edges are detected by measuring local intensity variation.

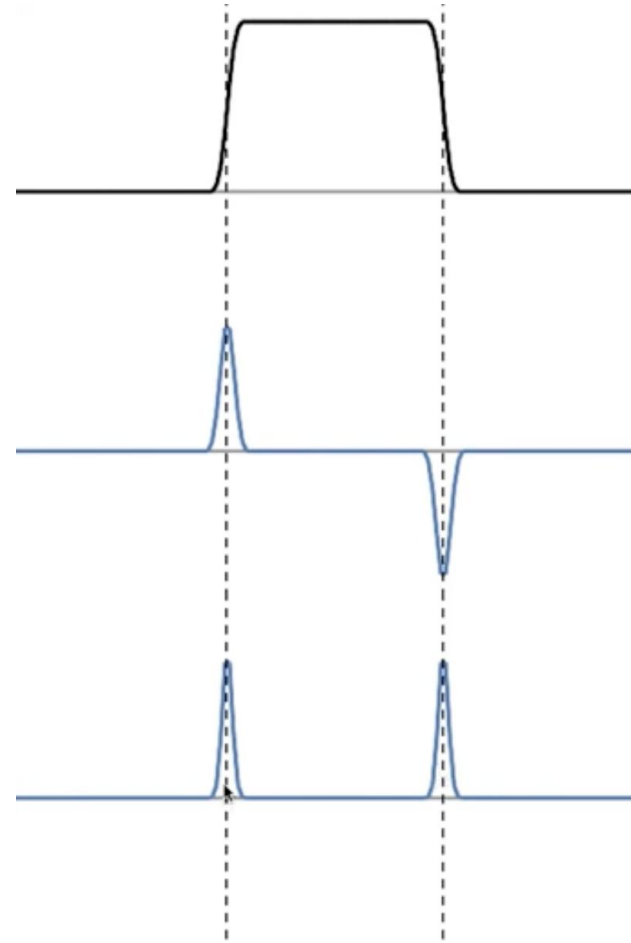
Edge detection

Edge Detection Using First Derivative:

$$f(x)$$

$$\text{First Derivative: } \frac{\partial f}{\partial x}$$

$$\text{First Derivative Absolute Value: } \left| \frac{\partial f}{\partial x} \right|$$



Edge Detection

- To detect edges, we compute:

- horizontal gradient $\frac{\partial I}{\partial x}$

- vertical gradient $\frac{\partial I}{\partial y}$

- From gradients, we get:

- edge strength:

$$S = \|\nabla I\| = \sqrt{\left(\frac{\partial I}{\partial x}\right)^2 + \left(\frac{\partial I}{\partial y}\right)^2}$$

- edge direction:

$$\theta = \tan^{-1} \left(\frac{\frac{\partial I}{\partial y}}{\frac{\partial I}{\partial x}} \right)$$

Edge Detection-Gradients

- In theory, an image is a continuous function $I(x, y)$.
- In practice, a digital image is a grid of discrete pixel values:
$$I[r, c], r = \text{row}, c = \text{column}$$
- This means:
 - derivatives do not exist analytically
 - they must be approximated numerically
- The goal is to estimate how intensity changes between neighboring pixels.
- In the discrete domain, derivatives are approximated using finite differences.

- Horizontal derivative

$$I_x(r, c) \approx I(r, c + 1) - I(r, c) \text{ or } \frac{I(r, c + 1) - I(r, c - 1)}{2}$$

- Vertical derivative

$$I_y(r, c) \approx I(r + 1, c) - I(r, c) \text{ or } \frac{I(r + 1, c) - I(r - 1, c)}{2}$$

- These measure:
 - change along columns $\rightarrow$ horizontal gradient
 - change along rows $\rightarrow$ vertical gradient
- In a nutshell: **gradients compare a pixel to its neighbors.**

Gradients as convolution filters

- Finite differences can be implemented as **convolutions**.
- **Prewitt operator :**
- The Prewitt operator:
 - combines:
 - differentiation in one direction
 - averaging in the perpendicular direction
 - Horizontal and vertical kernels:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \quad G_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

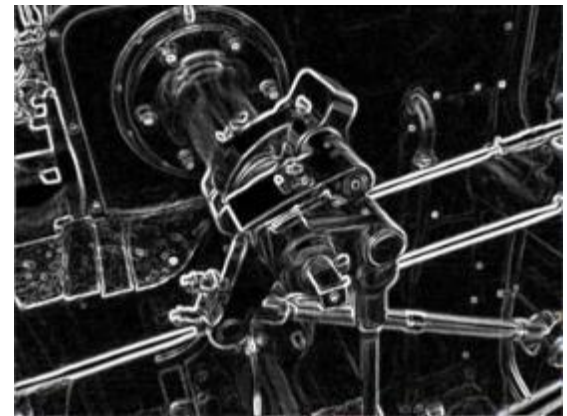
- Interpretation:
 - center difference $\rightarrow$ gradient
 - vertical / horizontal averaging $\rightarrow$ smoothing

Gradients as convolution filters

- The **Sobel operator** is a weighted version of Prewitt:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

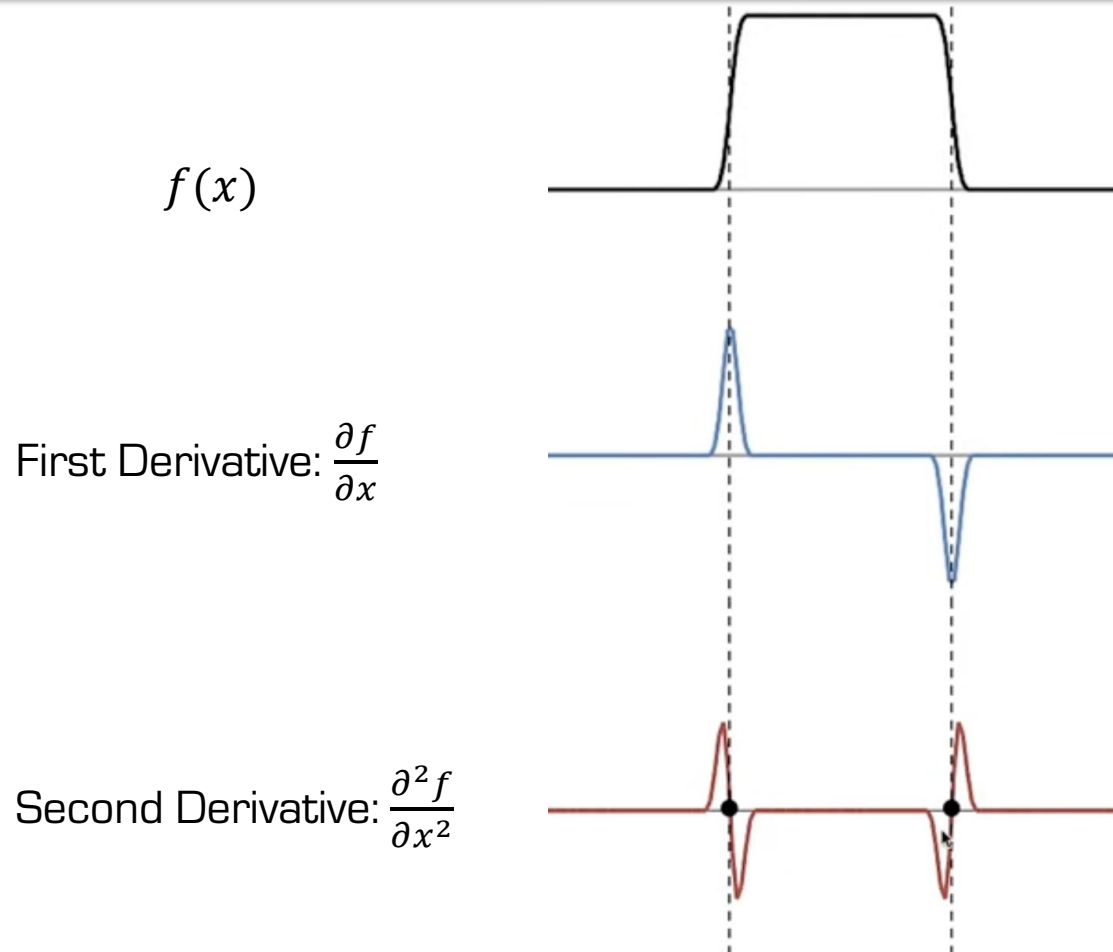
– central pixels get more weight



Edge Thresholding

- Standard: (Single Threshold T)
 $\|\nabla I(x, y)\| < T$ Definitely Not an Edge
 $\|\nabla I(x, y)\| \geq T$ Definitely an Edge
- Hysteresis Based: (Two Thresholds $T_0 < T_1$)
 $\|\nabla I(x, y)\| < T_0$ Definitely Not an Edge
 $\|\nabla I(x, y)\| \geq T_1$ Definitely an Edge
 $T_0 \leq \|\nabla I(x, y)\| < T_1$ Is an Edge if a
Neighboring Pixel is Definitely an Edge

Edge Detection: Second Derivative



- ✓ Edges correspond to **extrema of the first derivative**,
- ✓ which appear as **zero-crossings of the second derivative**.

Laplacian (∇^2) Operator

- The Laplacian combines second derivatives in both directions:

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}$$

- rotation invariant (no preferred direction)



Discrete Laplacian (∇^2) Operator

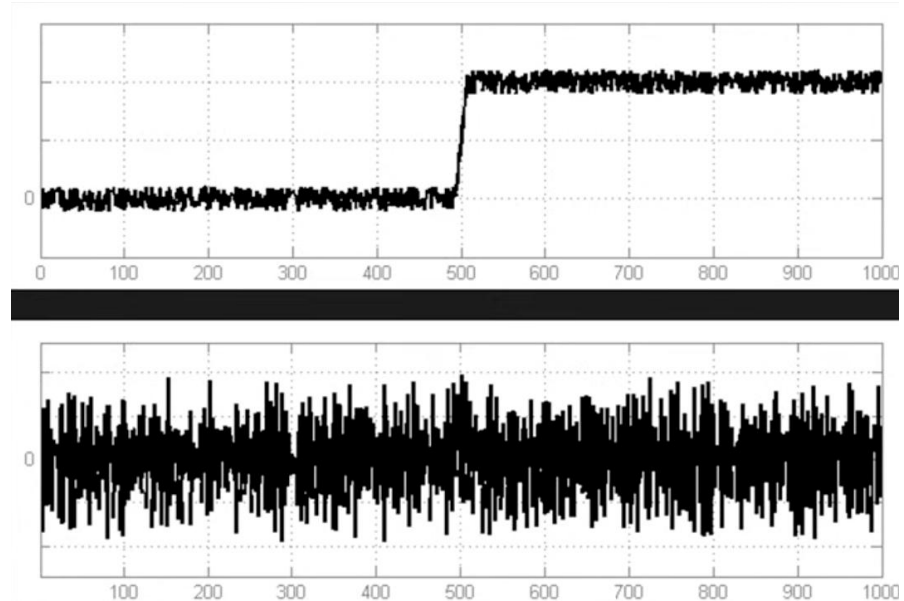
- In discrete images, the Laplacian is approximated by convolution.

Common 3×3 kernels:

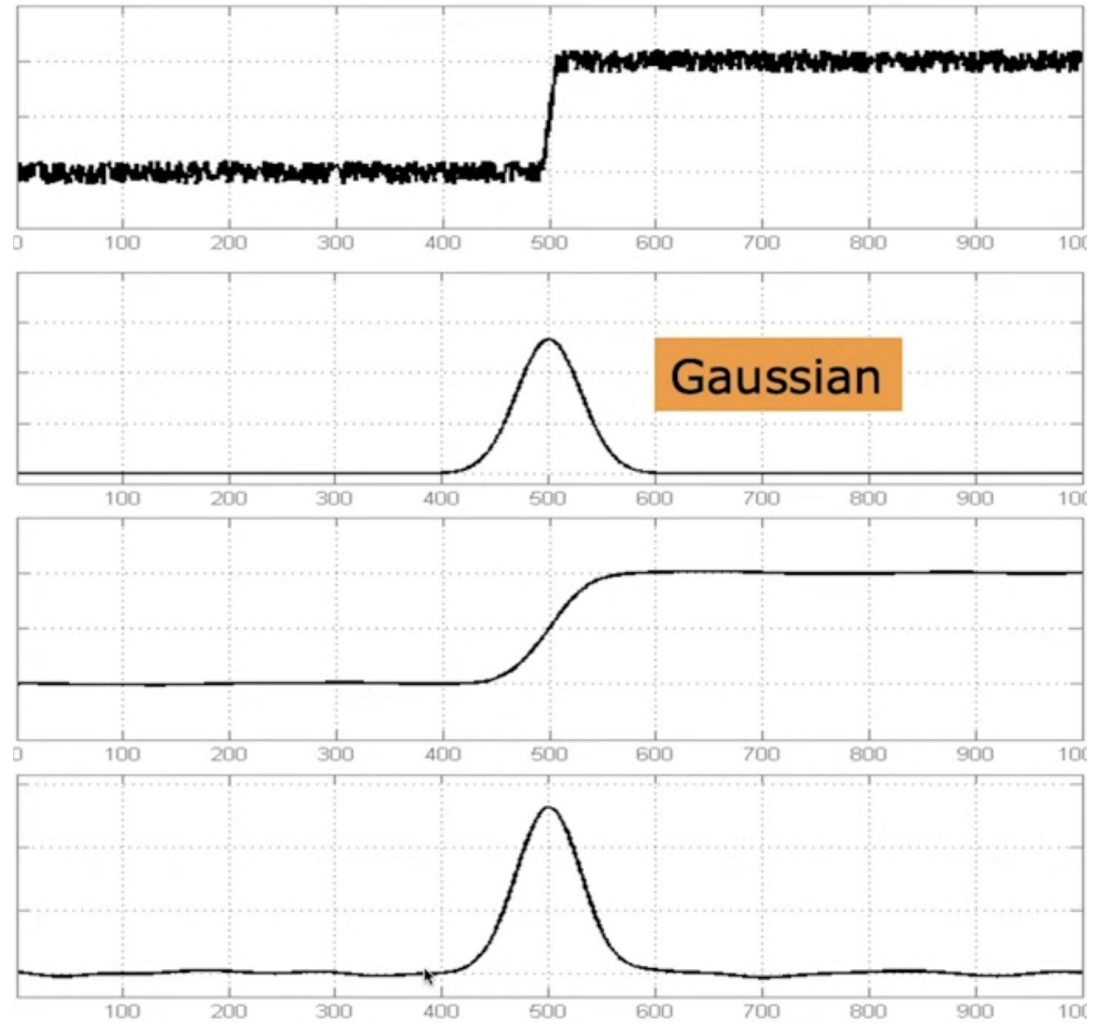
$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Discrete Laplacian (∇^2) Operator

- Second derivatives amplify noise even more than gradients.



Discrete Laplacian (∇^2) Operator



Discrete Laplacian (∇^2) Operator

- Second derivatives amplify noise even more than gradients.
- So in practice:
 - Smooth first (Gaussian)
 - Then apply Laplacian
- This leads to the Laplacian of Gaussian (LoG):
$$\nabla^2(G_\sigma * I)$$